

WIFI-OTA Upgrade Experiment

WIFI-OTA Upgrade Experiment

- 1. WIFI-OTA Overview
- 2. Core Concepts
 - 2.1 OTA Partition Layout
 - 2.2 Update Library Three-Step Process
 - 2.3 Streaming Download
 - 2.4 RGB LED Status Indicator
- 3. Hardware Dependencies
- 4. Project Architecture and Flow
- 5. Source Code Details
 - 5.1 Configuration and Global Variables
 - 5.2 `connectWiFi()` — WiFi Connection with Retry
 - 5.3 `setLed()` — RGB LED Wrapper
 - 5.4 `printProgress()` — Progress Bar Print
 - 5.5 Version Management (NVS)
 - 5.6 `checkupdate()` — OTA Main Flow
 - 5.7 `doOTA()` — Streaming Firmware Download and Write
 - 5.8 `compareVersion()` — Semantic Version Comparison
- 6. Operation Flow
 - 6.1 Tool Setup
 - 6.2 Operation Steps
 - 6.3 Verification Methods
- 7. FAQ

1. WIFI-OTA Overview

OTA (Over-The-Air) is a **core maintenance capability** for IoT devices—downloading new firmware over WiFi, writing it to the OTA partition in Flash, then automatically rebooting to run the new firmware. No physical USB cable needed, no JTAG programmer required; products can still be updated continuously after deployment.

Importance: OTA is the "lifeline" for mass-produced IoT products—without OTA, every bug fix or feature update requires manual disassembly and flashing, which is unacceptable. Smart home, industrial sensing, agricultural monitoring, and **all embedded devices requiring remote deployment** rely on OTA for continuous iteration.

This experiment powers on, fetches `version.txt` from the server via HTTP, compares it with the local version stored in NVS; if the server version is newer, it automatically downloads the `.bin` firmware and writes it to the OTA partition in Flash, then reboots to run the new firmware. During the upgrade process, the WS2812B RGB LED provides visual feedback: Red (upgrading) → Green (write complete) → Blue (successful reboot).

Typical Application Scenarios:

Application Type	Example
Batch device upgrades	Remote push of bug fixes after factory shipment

Application Type	Example
Unattended devices	Field sensors, rooftop weather stations without USB updates
Rapid iteration	WiFi OTA eliminates repeated USB plugging during development

2. Core Concepts

2.1 OTA Partition Layout

The ESP32-S3 default partition table contains two app partitions (`ota_0` + `ota_1`) and an `otadata` partition, which records which partition to boot from:

```
New firmware written to ota_1 → Update.end() verifies SHA256 → Updates otadata boot flag → ESP.restart() boots from ota_1
```

During the next upgrade, it alternately writes to the other partition (`ota_0`), implementing ping-pong upgrades.

2.2 Update Library Three-Step Process

Step	API	Function
① Initialize	<code>Update.begin(size)</code>	Erases target partition, prepares for writing
② Stream Write	<code>Update.write(buf, len)</code>	Write to Flash while downloading (no RAM usage)
③ Verify Complete	<code>Update.end()</code>	SHA256 verification + update boot flag, returns <code>true</code> on success

2.3 Streaming Download

ESP32 only has ~520KB SRAM, while firmware is typically >1MB. You cannot read the entire firmware into RAM first then write to Flash—you must read from HTTP stream and write via `Update.write()` simultaneously. The code uses `http.getStream().readBytes()` to read 2KB data chunks, write them, then continue to the next chunk until `size` reaches zero.

The `1.5MB` limit (`1024 * 1500`) provides safety margin for other partitions like SPIFFS and NVS.

2.4 RGB LED Status Indicator

The upgrade process provides visual feedback via the onboard WS2812B, no need to stare at the serial port:

LED Color	Stage	Meaning
Red	Before <code>doOTA()</code> starts	Firmware downloading and writing, do not power off

LED Color	Stage	Meaning
Green	When <code>update.end()</code> returns <code>true</code>	Flash write + SHA256 verification passed
Blue	After <code>saveNewVersion()</code>	Upgrade succeeded, about to reboot

3. Hardware Dependencies

Dependency	Description
Network Connection	WiFi 2.4GHz, WIFI-STA mode, HTTP server required on the same local network
WS2812B	GPIO 48 driven, upgrade status visualization
Flash Partition	OTA-compatible partition table required (e.g., <code>Minimal SPIFFS</code>), containing <code>ota_0</code> partition
Library Dependencies	<code>wiFi</code> , <code>HTTPClient</code> , <code>update</code> , <code>Preferences</code> (all built-in); <code>Adafruit_NeoPixel</code> (installed via library manager)
HTTP Server	PC hosts <code>version.txt</code> + <code>.bin</code> firmware, project provides <code>OTA_HTTP_Server.bat</code> + <code>OTA_Server.py</code> for one-click HTTP server startup (adapted from ESP-IDF OTA plugin : supports Range requests, CORS cross-domain, automatic LAN IP detection)

The onboard WS2812B is on GPIO 48; if using a different development board, modify the `RGB_PIN` macro.

4. Project Architecture and Flow

```
setup()
├─ prefs.begin("ota_config")           // Open NVS namespace (stores local version)
├─ pixels.begin() + clear()           // Initialize WS2812B RGB LED
├─ connectWiFi()                      // Connect to router (with retry)
└─ checkUpdate()                     // OTA main logic

checkUpdate():
├─ getLocalVersion()                 // Read local version from NVS, write "1.0.0" on
first run
├─ HTTP GET version.txt              // Fetch server version
├─ compareVersion(server, local) > 0? // Semantic version comparison
|   ├─ setLed(255,0,0) Red LED       // Upgrading, do not power off
|   └─ doOTA()                       // Download + write Flash + progress bar
|       ├─ HTTP GET firmware.bin (streaming, external WiFi client)
|       ├─ update.begin(totalSize) → write(buf) loop
|       ├─ printProgress() refreshes progress bar every 10%
|       └─ Returns true/false
```

```

|         └─ if ok:
|             └─ setLed(0,255,0) Green LED // write verification passed
|             └─ saveNewVersion(serverVer) // write new version to NVS
|             └─ setLed(0,0,255) Blue LED // Upgrade succeeded
|             └─ ESP.restart() // Soft reset to boot new firmware

loop()
└─ delay(10000) // Idle loop

```

5. Source Code Details

Complete source code: [Source code](#) → [Arduino](#) → [3.Network Communication](#) → [WiFi_OTA_Automatic](#) → [WiFi_OTA_Automatic.ino](#)

5.1 Configuration and Global Variables

The header includes WiFi, HTTP, OTA, NVS, and RGB LED five modules. `wifiClient client` is an external TCP client used for streaming firmware download in `doOTA()`. Version check includes 3 retries (consistent with ESP-IDF), tolerating network jitter. OTA server URL must match the IP printed by `OTA_Server.py`.

```

#include <WiFi.h>
#include <WiFiClient.h>
#include <Update.h>
#include <Preferences.h>
#include <HTTPClient.h>
#include <Adafruit_NeoPixel.h>

// ===== Config Section / Configuration =====
const char* WIFI_SSID = "Yahboom2";
const char* WIFI_PASS = "yahboom890729";

// OTA server URL - Replace with IP printed by OTA_Server.py after running it
#define VERSION_URL    "http://192.168.11.183:8080/version.txt"
#define FIRMWARE_URL   "http://192.168.11.183:8080/wifi_ota_automatic.ino.bin"

#define DEFAULT_VERSION "1.0.0" // Default version in NVS
#define RGB_PIN         48      // WS2812B pin
#define LED_NUM          1      // Number of LEDs
#define OTA_RETRY_MAX    3      // Version check retry count
#define OTA_RETRY_WAIT   2000   // Retry interval ms

Preferences prefs; // NVS storage object
WiFiClient client; // TCP client
Adafruit_NeoPixel pixels(LED_NUM, RGB_PIN, NEO_GRB + NEO_KHZ800);

```


5.4 `printProgress()` — Progress Bar Print

Draws an ASCII progress bar on serial port to visually show firmware download progress.

```
void printProgress(int percent) {
    const int barwidth = 40;                // Progress bar width
    int filled = barwidth * percent / 100;    // Number of filled blocks
    Serial.print("[");
    for (int i = 0; i < barwidth; i++) {
        if (i < filled)      Serial.print("=");
        else if (i == filled) Serial.print(">");
        else                 Serial.print(" ");
    }
    Serial.printf("] %3d%%\n", percent);      // Percentage + newline
}
```

Output: [=====>] 25%

> marks the current progress position, completed portions use =, remaining uses space. 40 characters wide, refreshes every 10%.

5.5 Version Management (NVS)

NVS (Non-Volatile Storage) persists version numbers in Flash, surviving power loss. Writes default version 1.0.0 on first run.

```
String getLocalVersion() {
    if (!prefs.iskey("ver")) {                // Write default if key missing
        prefs.putString("ver", DEFAULT_VERSION);
    }
    return prefs.getString("ver");            // Read version from NVS
}

void saveNewVersion(String v) {
    prefs.putString("ver", v);                // Write new version to NVS
    Serial.println("New version saved to Flash");
}
```

5.6 `checkUpdate()` — OTA Main Flow

Fetches server version, compares with local, decides whether to trigger OTA. Consistent with ESP-IDF, includes 3 retries for network jitter tolerance.

```
void checkUpdate() {
    if (WiFi.status() != WL_CONNECTED) return; // Skip if WiFi not connected

    String local = getLocalVersion();
    Serial.printf("Local version: %s\n", local.c_str());

    HTTPClient http;
    http.setFollowRedirects(HTTPC_STRICT_FOLLOW_REDIRECTS);
}
```

```

http.setConnectTimeout(5000);           // 5s connect timeout

bool gotVersion = false;
String serverVer;
for (int retry = 0; retry < OTA_RETRY_MAX; retry++) {
    if (!http.begin(VERSION_URL)) {
        Serial.printf("Version check failed (retry %d/%d)\n", retry + 1, OTA_RETRY_MAX);
    } else {
        int code = http.GET();
        if (code == HTTP_CODE_OK) {
            serverVer = http.getString();           // Read server version string
            serverVer.trim();                       // Trim whitespace
            gotVersion = true;
            http.end();
            break;
        }
        Serial.printf("HTTP code: %d (retry %d/%d)\n", code, retry + 1, OTA_RETRY_MAX);
        http.end();
    }
    delay(OTA_RETRY_WAIT);                       // wait before retry
}

if (!gotVersion) {
    Serial.println("Version check failed after all retries");
    return;
}

Serial.printf("Server version: %s\n", serverVer.c_str());

if (compareVersion(serverVer, local) > 0) {       // Server version is newer
    Serial.println("New version found! Start upgrading...");
    setLed(255, 0, 0);                           // Red LED – updating, don't power off
    if (doOTA()) {
        setLed(0, 255, 0);                       // Green LED – firmware written
        delay(1000);
        saveNewVersion(serverVer);
        setLed(0, 0, 255);                       // Blue LED – upgrade succeeded
        Serial.println("Upgrade succeeded! Restarting...");
        delay(2000);
        ESP.restart();                           // Soft reset, boot new firmware
    } else {
        Serial.println("Upgrade failed");
    }
} else {
    Serial.println("Already the latest version");
}
}

```

`OTA_RETRY_MAX=3` is consistent with ESP-IDF OTA routine retry count. `OTA_RETRY_WAIT=2000ms` gives the server enough processing time. After `http.GET()` fetches `version.txt` content, `trim()` removes line breaks.

5.7 doOTA() — Streaming Firmware Download and Write

Four-step flow: HTTP GET to fetch firmware → `Update.begin()` to erase partition → 2KB chunked streaming write to Flash (refresh progress bar every 10%) → `Update.end()` SHA256 verification. Never loads complete firmware into RAM.

```
bool doOTA() {
    HTTPClient http;
    http.setFollowRedirects(HTTPC_STRICT_FOLLOW_REDIRECTS);

    if (!http.begin(client, FIRMWARE_URL)) {           // Streaming download via external
WiFiClient
        Serial.println("Failed to connect to firmware server");
        return false;
    }

    int code = http.GET();
    if (code != HTTP_CODE_OK) {
        Serial.printf("Firmware download failed, HTTP code: %d, error: %s\n",
                      code, http.errorToString(code).c_str());
        http.end();
        return false;
    }

    int totalSize = http.getSize();                   // Firmware size from Content-Length
    Serial.printf("Firmware size: %d bytes\n", totalSize);

    if (totalSize > 1024 * 1500) {                     // 1.5MB safety limit
        Serial.println("Firmware too large, partition cannot fit it");
        http.end();
        return false;
    }

    Update.begin(totalSize);                          // Init OTA write, erase partition

    uint8_t buf[2048];                               // 2KB buffer, flash page-aligned
    int readed, downloaded = 0;
    int lastPercent = -1;                             // Last printed percent, avoid redundant
    print

    while (http.connected() && totalSize > 0) {        // Streaming download loop
        readed = http.getStream().readBytes(buf, sizeof(buf));
        if (readed > 0) {
            Update.write(buf, readed);                // Write to flash
            totalSize -= readed;                       // Remaining bytes
            downloaded += readed;
        }
        int percent = (totalSize <= 0) ? 100 : (downloaded * 100 / (downloaded + totalSize));
        if (percent != lastPercent && percent % 10 == 0) { // Refresh every 10%
            printProgress(percent);
            lastPercent = percent;
        }
    }
}
```



```

}
printProgress(100);                // Ensure 100% shown
Serial.println();

bool ok = update.end();            // Finalize, SHA256 verify
http.end();
return ok;
}

```

`totalSize` tracks remaining bytes, `downloaded` tracks downloaded bytes. `lastPercent` prevents printing the same percentage repeatedly. Calls `printProgress()` every 10%, forces printing `100%` after loop ends to prevent incomplete progress bar if loop exits early.

5.8 `compareVersion()` — Semantic Version Comparison

Uses `sscanf` to parse `MAJOR.MINOR.PATCH` three-part numbers, comparing segment by segment. Return value >0 means A is newer than B, <0 means A is older, 0 means equal.

```

int compareVersion(String a, String b) {
    int va[3] = {0}, vb[3] = {0};          // major, minor, patch
    sscanf(a.c_str(), "%d.%d.%d", &va[0], &va[1], &va[2]);
    sscanf(b.c_str(), "%d.%d.%d", &vb[0], &vb[1], &vb[2]);

    if (va[0] > vb[0]) return 1;            // A major > B major
    if (va[0] < vb[0]) return -1;
    if (va[1] > vb[1]) return 1;            // A minor > B minor
    if (va[1] < vb[1]) return -1;
    if (va[2] > vb[2]) return 1;            // A patch > B patch
    if (va[2] < vb[2]) return -1;
    return 0;                              // Versions equal
}

```

Compares `MAJOR.MINOR.PATCH` segment by segment. Return value >0 means A is newer than B, <0 means A is older, 0 means equal.

6. Operation Flow

6.1 Tool Setup

Arduino Flashing Flow: See `Arduino → 1.Before Development → 2.Build and Flash`.

① Install Dependent Libraries

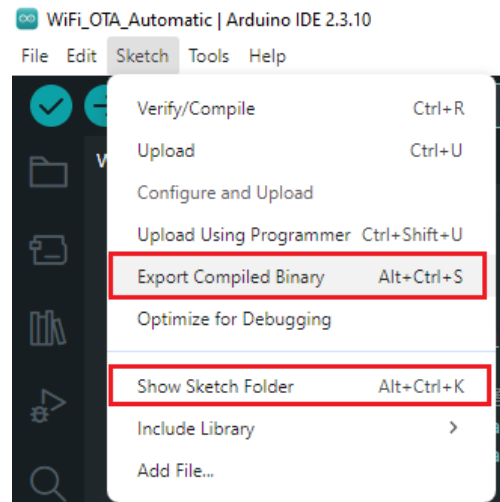
Arduino IDE → Tools → Manage Libraries → Search `Adafruit NeoPixel` → Install.

② Modify WiFi SSID and Password

Change `WIFI_SSID` and `WIFI_PASS` in the code to your actual router credentials.

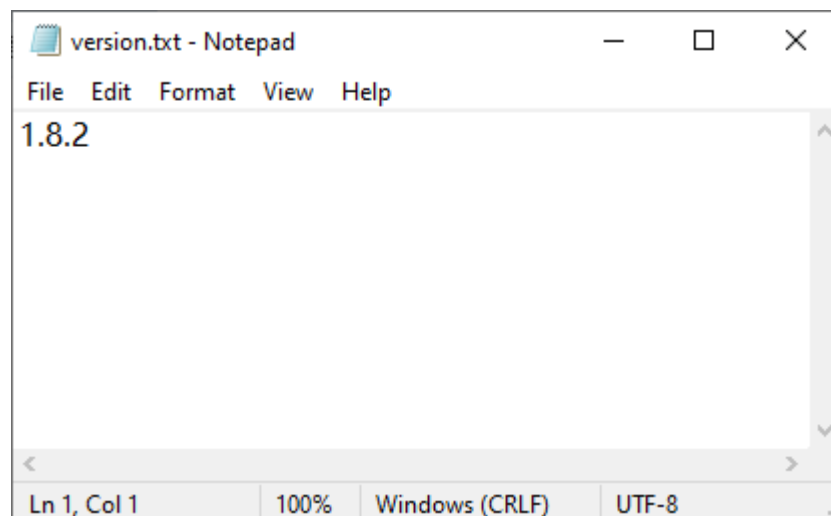
③ Compile and Export bin Firmware

Arduino IDE → Sketch → Export Compiled Binary (or shortcut `Ctrl+Alt+S`). After compilation, the `.bin` file is located in `build/esp32.esp32.esp32s3/` within the project directory, named `wifi_OTA_Automatic.ino.bin`.



Arduino > 3.Network Communication > WiFi_OTA_Automatic > build > esp32.esp32.esp32s3				
Name	Date modified	Type	Size	
boot_app0.bin	8/5/2026 8:13 PM	BIN File	8 KB	
build.options.json	8/7/2026 10:54 AM	JSON Source File	1 KB	
flash_args	8/7/2026 10:54 AM	File	1 KB	
partitions.csv	8/5/2026 8:13 PM	XLS 工作表	1 KB	
sdkconfig	8/5/2026 8:13 PM	File	141 KB	
WiFi_OTA_Automatic.ino.bin	8/7/2026 10:54 AM	BIN File	1,008 KB	
WiFi_OTA_Automatic.ino.bootloader.bin	8/7/2026 10:54 AM	BIN File	20 KB	
WiFi_OTA_Automatic.ino.elf	8/7/2026 10:54 AM	ELF File	13,747 KB	
WiFi_OTA_Automatic.ino.map	8/7/2026 10:54 AM	MAP File	16,538 KB	
WiFi_OTA_Automatic.ino.merged.bin	8/7/2026 10:54 AM	BIN File	4,096 KB	
WiFi_OTA_Automatic.ino.partitions.bin	8/7/2026 10:54 AM	BIN File	3 KB	

- Copy this `.bin` file to the `OTA_UPDATA` folder, together with `version.txt`
- In `version.txt`, write a version number higher than `1.0.0` (e.g., `1.8.2`)



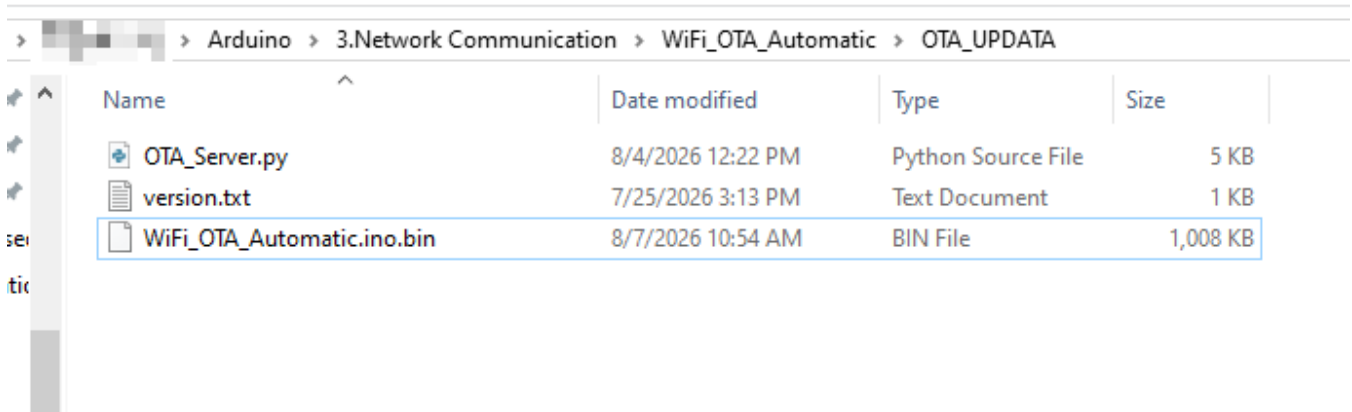
④ Initial Flash

Use USB to flash the current code to ESP32-S3 (local version 1.0.0).

6.2 Operation Steps

Step 1: Start HTTP Server

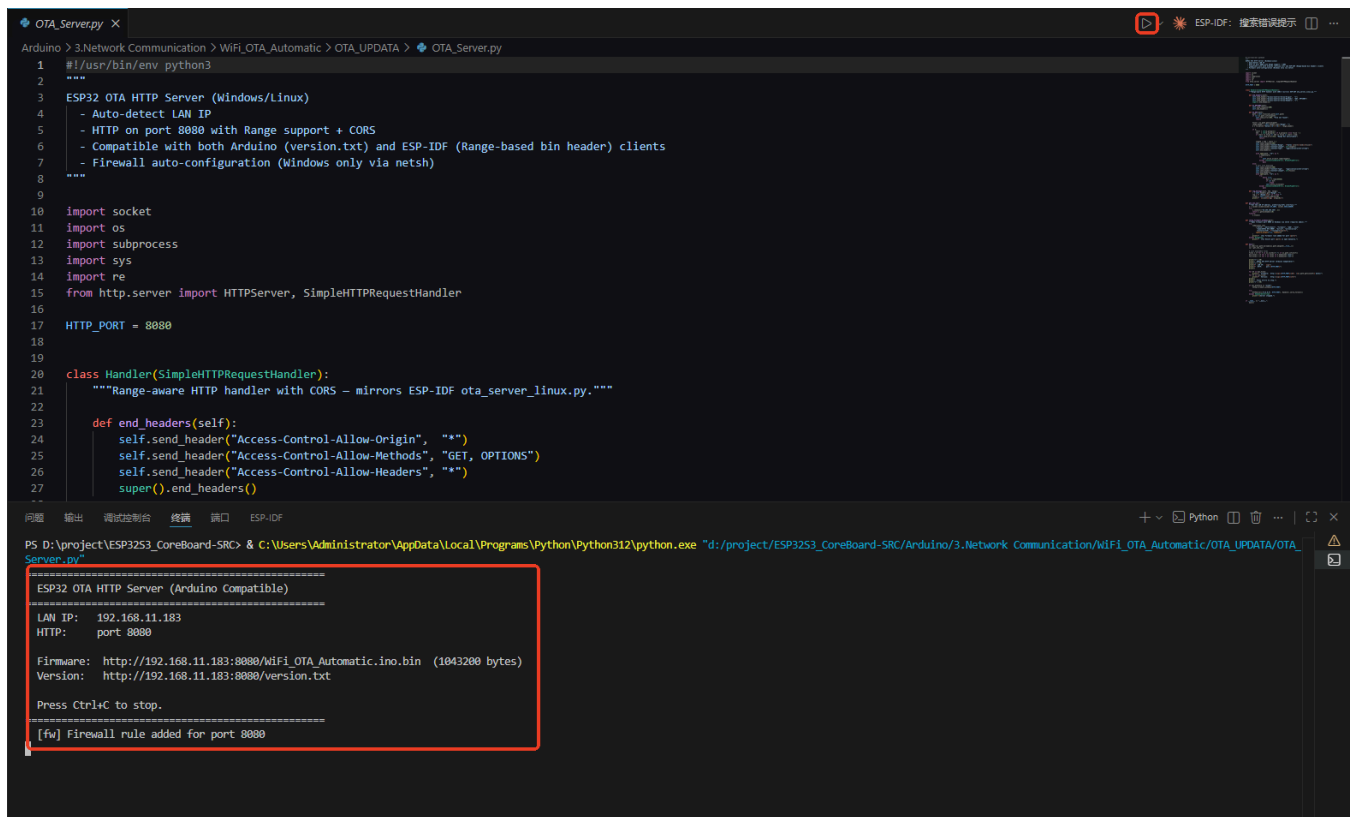
Confirm the `.bin` firmware and `version.txt` are in the `OTA_UPDATA` folder (see Step ③ in 6.1).



Double-click to start `OTA_Server.py`.

- Detect Python environment
- Get local LAN IP
- Configure Windows Firewall (open port 8080)
- Run `OTA_Server.py` HTTP server (adapted from **ESP-IDF OTA plugin**, supports **Range requests**, **CORS cross-domain**)

The server prints the complete URLs for `version.txt` and `.bin` firmware:



```
PS D:\project\ESP32S3_CoreBoard-SRC> & C:\Users\Administrator\AppData\Local\Programs\Python\Python312\python.exe "d:/project/ESP32S3_CoreBoard-SRC/Arduino/3.Network Communication/WiFi_OTA_Automatic/OTA_UPDATA/OTA_Server.py"

=====
ESP32 OTA HTTP Server (Arduino Compatible)
=====
LAN IP: 192.168.11.183
HTTP: port 8080

Firmware: http://192.168.11.183:8080/WiFi_OTA_Automatic.ino.bin (1043200 bytes)
Version: http://192.168.11.183:8080/version.txt

Press Ctrl+C to stop.
=====
[fw] Firewall rule added for port 8080
```

`ota_server.py` is adapted from ESP-IDF OTA plugin `ota_server_linux.py`, supports HTTP Range requests (206 Partial Content), providing compatibility for future extracting version numbers directly from firmware binary header. If Windows Firewall popup appears, click "Allow access". If Python is not installed prompt appears, install Python 3 first.

Step 2: Modify Server IP, Initial Flash

Open Arduino IDE, change the IP address inside `VERSION_URL` and `FIRMWARE_URL` to the actual IP output by the python script; after compilation, flash to ESP32-S3 via USB.

It is recommended to use WiFi network preferably, try to avoid using ethernet cable; if needed, you can disable local ethernet adapter.

Step 3: Observe OTA Auto Upgrade

After flashing, open Serial Monitor (baud rate 115200), ESP32 automatically connects to WiFi and checks version. If server version is newer, it automatically downloads firmware and prints progress bar:

```
Connecting WiFi: Yahboom2 (Attempt 1)
WiFi Connected successfully
Device IP: 192.168.11.229
Local version: 1.0.0
Server version: 1.8.2
New version found! Start upgrading...
Firmware size: 1048576 bytes
[=====] 100%
New version saved to Flash
Upgrade succeeded! Restarting...
```

```
Connecting WiFi: Yahboom2 (Attempt 1)
...
WiFi Connected successfully
Device IP: 192.168.11.229
Version URL: http://192.168.11.183:8080/version.txt
Local version: 1.0.0
Server version: 1.8.2
New version found! Start upgrading...
Firmware size: 1043200 bytes
[>] 0%
[====>] 10%
[=====>] 20%
[=====>] 30%
[=====>] 40%
[=====>] 50%
[=====>] 60%
[=====>] 70%
[=====>] 80%
[=====>] 90%
[=====>] 100%
[=====>] 100%

New version saved to Flash
Upgrade succeeded! Restarting...
```

The onboard RGB LED changes in sequence: **Red (downloading)** → **Green (verification passed)** → **Blue (about to reboot)**.

The entire process the onboard RGB LED changes: **Red** → **Green** → **Blue** → **Reboot**.

Step 4: Verify Upgrade Success

After reboot, Serial prints `Local version: 1.8.2`, matching server version, and no longer triggers upgrade (prints `Already the latest version`). If firmware runtime behavior differs from old version (you can intentionally change print content in `setup()`), it also confirms the upgrade took effect.

```
WiFi Connected successfully
Device IP: 192.168.11.229
Version URL: http://192.168.11.183:8080/version.txt
Local version: 1.8.2
Server version: 1.8.2
Already the latest version
```

6.3 Verification Methods

Operation	Expected Result
Server version > Local version	Red→Green→Blue LED, auto download and reboot
Server version = Local version	Prints "Already the latest version", no upgrade
Server unreachable	After 3 retries, prints failure message, skips upgrade (consistent with ESP-IDF retry strategy)
Firmware server unreachable	Version check passes but <code>doOTA()</code> returns false

7. FAQ

Symptom	Cause	Solution
<code>update.begin()</code> fails	Partition table doesn't support OTA	Select board option with OTA partition in Arduino IDE (e.g., <code>Minimal SPIFFS</code>)
<code>update.end()</code> verification fails	Firmware download incomplete or corrupted	Check network stability, reduce WiFi interference
Still running old firmware after reboot	otadata not updated	Confirm <code>update.end()</code> returned <code>true</code>

Symptom	Cause	Solution
Version comparison always triggers upgrade	<code>version.txt</code> trailing newline causes sscanf parsing anomaly	Code already handles this with <code>serverVer.trim()</code>
<code>http.GET()</code> returns -1 or retry 3 times fails	Server not started or wrong IP	Verify by browsing to <code>http://<IP>:8080/version.txt</code> ; confirm using <code>OTA_Server.py</code> not <code>python -m http.server</code>
Compilation error: <code>Adafruit_NeoPixel.h</code> not found	Library not installed	Search and install in Arduino IDE Library Manager